1. Understand hashCode() and equals() Contract for HashSet Uniqueness (Interview Notes)

Why Does HashSet Need hashCode() and equals()?

A HashSet stores **only unique elements**.

To determine whether an object already exists, it uses:

1. **hashCode()** → Finds the bucket where the object should be stored.
2. **equals()** → Checks whether two objects are actually equal.

Both methods work together to prevent duplicates.

Example

```
class Student {

    int id;

    Student(int id) {
        this.id = id;
    }

    @Override
    public int hashCode() {
        return id;
    }

    @Override
    public boolean equals(Object obj) {
        Student s = (Student) obj;
        return this.id == s.id;
    }
}
```

```
HashSet set = new HashSet<>();

set.add(new Student(1));
set.add(new Student(1));

System.out.println(set.size());
```

Output

```
1
```

Without overriding both methods:

```
2
```

Because Java treats them as different objects.

How HashSet Checks Duplicates

```
New Object

↓

hashCode()

↓

Find Bucket

↓

equals()

↓

Duplicate?
```

```
Yes → Ignore

No → Add
```

Important Interview Points ⭐

- HashSet uses **both hashCode() and equals()** to ensure uniqueness.
- hashCode() determines **where** an object is stored (bucket).
- equals() determines **whether two objects are logically equal**.
- If two objects are equal according to equals(), they **must return the same hashCode()**.
- Overriding only one of these methods can cause incorrect duplicate detection.
- Hash collisions are normal; equals() resolves them.
- Always override **both methods together** when storing custom objects in a HashSet or using them as HashMap keys.

Tricky Interview Questions

Q1. Why does HashSet use both hashCode() and equals()?

Answer:

- hashCode() → Locate the bucket.
- equals() → Verify logical equality.

Q2. Can two different objects have the same hashCode()?

Answer: Yes.

This is called a **hash collision**.

Q3. If two objects are equal, must their hashCode() be the same?

Answer: Yes.

This is a fundamental Java contract.

Q4. What happens if you override equals() but not hashCode()?

Answer:

HashSet may store duplicate logical objects because they end up in different buckets.

Q5. What happens if two objects have the same hashCode() but equals() returns false?

Answer:

Both objects are stored because they are not logically equal.

Q6. Which method is called first by HashSet?

Answer:

hashCode() first, then equals() (if required).

Interview Summary ⭐

- **hashCode() and equals() work together to maintain uniqueness in HashSet.**
- hashCode() identifies the bucket; equals() confirms equality.
- **Equal objects must produce the same hash code**, but the same hash code does not necessarily mean objects are equal.
- Override **both methods together** for custom classes.
- This contract is also critical for HashMap keys.

> **Interview One-Liner:**
>
> **"HashSet uses hashCode() to locate the bucket and equals() to verify equality, ensuring that logically duplicate objects are not stored."**

---

2. Use TreeSet for Sorted Unique Elements (Interview Notes)

What is TreeSet?

TreeSet is an implementation of the **Set** interface that stores **unique elements in sorted order**.

It automatically sorts elements according to their **natural ordering** or a provided **Comparator**.

```
TreeSet set = new TreeSet<>();

set.add(30);
set.add(10);
set.add(20);
set.add(10);

System.out.println(set);
```

Output

```
[10, 20, 30]
```

Duplicates are removed and elements are sorted.

Why Use TreeSet?

Use TreeSet when:

- Duplicate elements are not allowed.
- Elements must always remain sorted.
- Fast searching with automatic ordering is required.

Important Interview Points ⭐

- TreeSet implements the **Set** interface.
- Stores **unique elements only**.
- Maintains elements in **ascending sorted order** by default.
- Internally implemented using a **Red-Black Tree (self-balancing BST)**.
- Does **not allow duplicate elements**.
- Does **not allow null** (in modern Java, adding null throws NullPointerException).
- Elements must implement **Comparable** or a **Comparator** must be supplied.
- Operations like add(), remove(), and contains() have **O(log n)** time complexity.

TreeSet vs HashSet

Feature	HashSet	TreeSet
Duplicates	❌	❌
Order	No guarantee	Sorted
Internal Structure	Hash Table	Red-Black Tree
add()	O(1) average	O(log n)
contains()	O(1) average	O(log n)

Tricky Interview Questions

Q1. Why is TreeSet slower than HashSet?

Answer:

Because TreeSet maintains sorted order using a **Red-Black Tree**, making operations **O(log n)** instead of **O(1)** average.

Q2. Does TreeSet allow duplicates?

Answer: No.

Q3. Does TreeSet maintain insertion order?

Answer: No.

It maintains **sorted order**, not insertion order.

Q4. Can TreeSet store null?

Answer: No.

Adding null results in a NullPointerException because TreeSet must compare elements.

Q5. How does TreeSet sort custom objects?

Answer:

By implementing the **Comparable** interface or by providing a **Comparator**.

Q6. Which is better for fast searching: HashSet or TreeSet?

Answer:

- **HashSet** → Faster (**O(1)** average).
- **TreeSet** → Slower (**O(log n)**), but keeps data sorted.

Interview Summary ⭐

- **TreeSet stores unique elements in automatically sorted order.**
- Internally uses a **Red-Black Tree**, giving **O(log n)** performance for major operations.
- Does **not allow duplicates** or null values.
- Uses **Comparable** or **Comparator** for sorting custom objects.
- Choose **TreeSet** when sorted data is required; choose **HashSet** when maximum lookup performance is more important than ordering.

> **Interview One-Liner:**
>
> **"TreeSet stores unique elements in sorted order using a Red-Black Tree, providing O(log n) operations while automatically maintaining element order."**